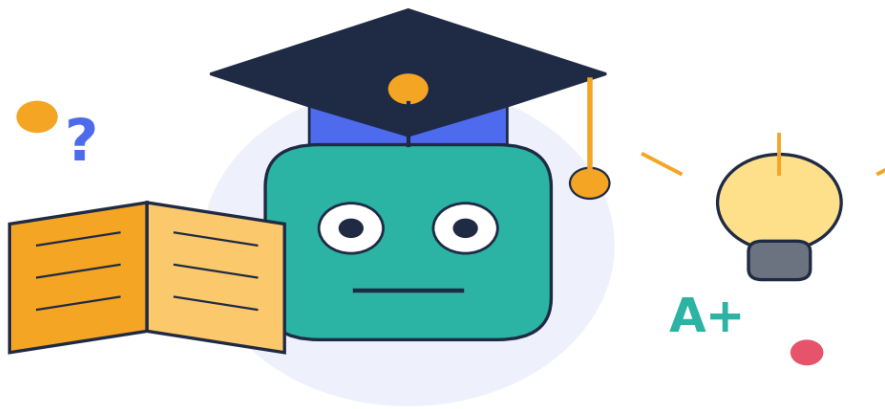


# EDUGENIE

Google Gemini Powered Learning Assistant



*EduGenie: your AI study companion*

## PROJECT REPORT

Submitted in partial fulfillment of the requirements for the course

Submitted by

**[ S.ROSHINI ]**

Register No: [212506781]

Department of [BCA]

[T] S COLLEGE OF ARTS AND SCIENCE]

Academic Year 2026-2027

# CERTIFICATE

This is to certify that the project report entitled “EduGenie: Google Gemini Powered Learning Assistant” is a bonafide work carried out by [S.ROSHINI], Register No. [212506781], of the Department of [DEPARTMENT], [COLLEGE NAME], during the academic year 2026-2027.

The project has been completed under the guidance of [GUIDE NAME] and is submitted as part of the academic requirements of the course.

Place: [PLACE]

Date: [DATE]

Project Guide: S.ROSHINI

Head of Department: SIVARANJANI

## PROJECT TEAM

**Team Leader:** S.ROSHINI

**Team Member:** R. PRADEEPA

# DECLARATION

I hereby declare that the project entitled “EduGenie: Google Gemini Powered Learning Assistant” is my original academic project work carried out for the purpose of learning and project submission.

The information, design, implementation ideas, and documentation presented in this report have been prepared for educational use. Wherever external technical information has been used, the relevant source has been acknowledged.

I understand that the project should be demonstrated according to the requirements given by my institution and project coordinator.

Name: [S.ROSHINI]

Register No: [212506781]

# ACKNOWLEDGEMENT

I express my sincere thanks to my project guide, faculty members, and department for their valuable guidance and support during the development of the EduGenie project.

I am thankful to my college for providing the learning environment and resources required to understand artificial intelligence, application development, and API integration.

I also thank my classmates, friends, and family members for their encouragement and support. Their suggestions helped me improve the project idea and presentation.

Finally, I acknowledge the official Google Gemini API documentation used to understand current API integration concepts and development practices.

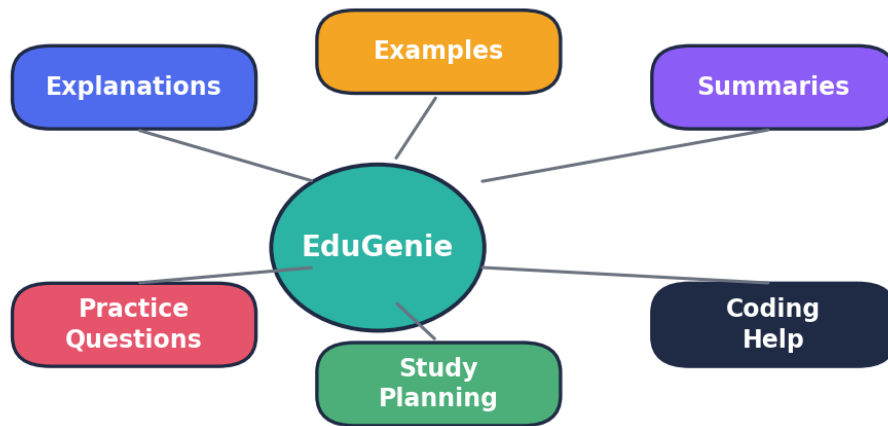
# ABSTRACT

EduGenie is an AI-powered learning assistant designed to help students understand academic topics in a simple and interactive way. The system uses Google Gemini through an API to generate explanations, examples, summaries, practice questions, and study assistance based on the student's prompt.

The main purpose of EduGenie is to make learning more accessible and personalized. Instead of searching through multiple sources for every small doubt, a student can enter a question and receive a structured response. The application can be designed with a simple user interface so that students can interact with it like a study chatbot.

The proposed system includes modules such as question answering, topic explanation, summarization, quiz generation, study planning, and basic coding assistance. The project demonstrates how generative AI can be integrated into an educational application using an API.

EduGenie is intended as a learning support tool. Students should verify important academic information with textbooks, teachers, and trusted sources.



*Figure: How EduGenie helps a student*

# TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives of the Project
4. Existing System
5. Proposed System
6. System Requirements
7. System Design and Architecture
8. Modules of EduGenie
9. Gemini API Integration
10. Implementation
11. User Interface and Workflow
12. Testing
13. Advantages and Limitations
14. Future Enhancements
15. Conclusion and References

# 1. INTRODUCTION

Artificial Intelligence (AI) is increasingly used to support education, communication, programming, research, and other everyday activities. Generative AI systems can understand natural-language prompts and produce useful responses. EduGenie applies this idea to student learning.

EduGenie acts as a digital learning assistant. A student can enter a topic such as “Explain binary search in simple words” or “Give five practice questions on management accounting.” The application sends the request to the Gemini API and displays the generated response.

The project focuses on simplicity. Students may need short definitions, step-by-step explanations, examples, revision questions, and summaries. EduGenie combines these common requirements in one application.

The application can be implemented as a web or desktop project. A typical web version can contain a frontend for entering questions and a backend service that securely communicates with the Gemini API.

According to Google’s official documentation, the Gemini API supports text generation and multimodal capabilities, and current documentation recommends the Interactions API for new projects.

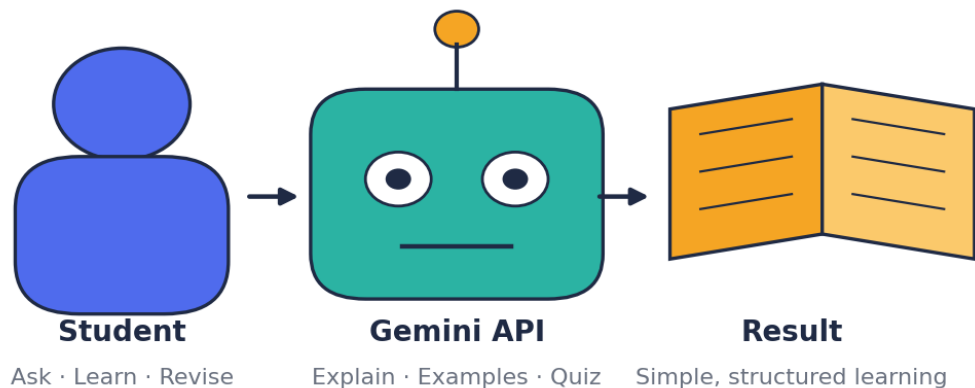


Figure 1: EduGenie at a glance (Student → Gemini API → Result)

## 2. PROBLEM STATEMENT

Students often face difficulty when they do not understand a topic during self-study. Textbooks may contain detailed explanations, while students may need a shorter and easier explanation. Searching many websites can also take time and may produce information that is difficult to understand.

The problem addressed by EduGenie is the need for a simple educational assistant that can respond to student questions in natural language and adapt the response to the requested level of detail.

Major problems considered in the project are:

- Difficulty in understanding complex academic concepts.
- Need for quick explanations and examples.
- Difficulty in creating practice questions for revision.
- Time required to search multiple resources.
- Need for a single interface for common study-support tasks.
- Need for beginner-friendly explanations in simple language.

EduGenie does not replace teachers or textbooks. It is designed as an additional learning support system.



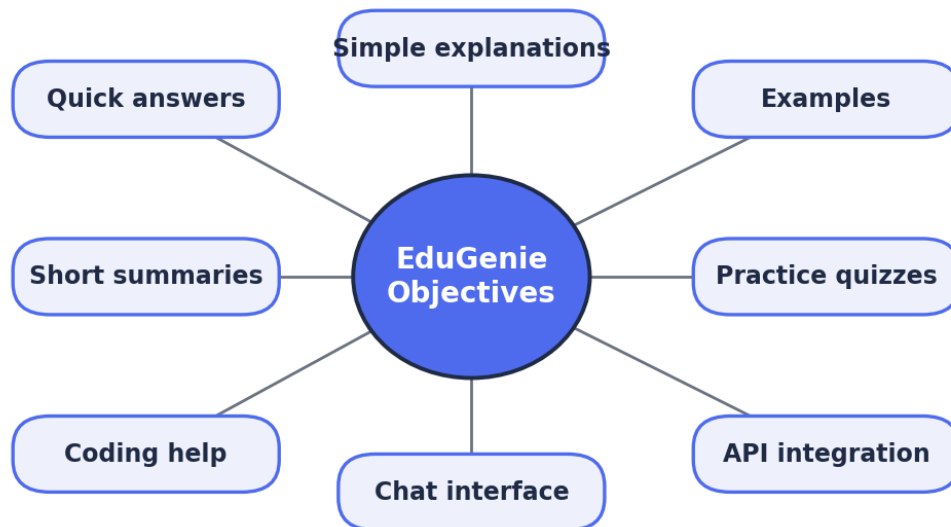
Figure 2: Common problems students face during self-study

### 3. OBJECTIVES OF THE PROJECT

The major objective of EduGenie is to develop an easy-to-use AI learning assistant that helps students learn academic concepts through natural-language interaction.

- To provide quick answers to student questions.
- To explain difficult topics in simple language.
- To generate examples for better understanding.
- To create short summaries for revision.
- To generate practice questions and quizzes.
- To provide basic programming explanations and examples.
- To provide a simple conversational learning interface.
- To demonstrate API integration with a generative AI model.
- To improve student access to personalized study support.

The project also provides practical exposure to frontend development, backend communication, API authentication, prompt design, error handling, and software testing.



*Figure 3: Objectives of EduGenie*

## 4. EXISTING SYSTEM

In a traditional learning approach, students use textbooks, classroom notes, search engines, educational websites, videos, and direct help from teachers. These methods are useful but may not always provide an immediate explanation in the exact format required by a student.

For example, a student may want a two-line definition, a five-mark answer, a step-by-step solution, or a set of practice questions. The student may need to search and reorganize information manually.

Limitations of the existing approach:

- Information is distributed across different resources.
- Students may spend additional time searching.
- Some explanations may be too advanced for beginners.
- Creating personalized practice material can take time.
- Different resources may use different styles and terminology.

The existing system is therefore not replaced completely; EduGenie adds an AI-based assistance layer that can provide conversational support.

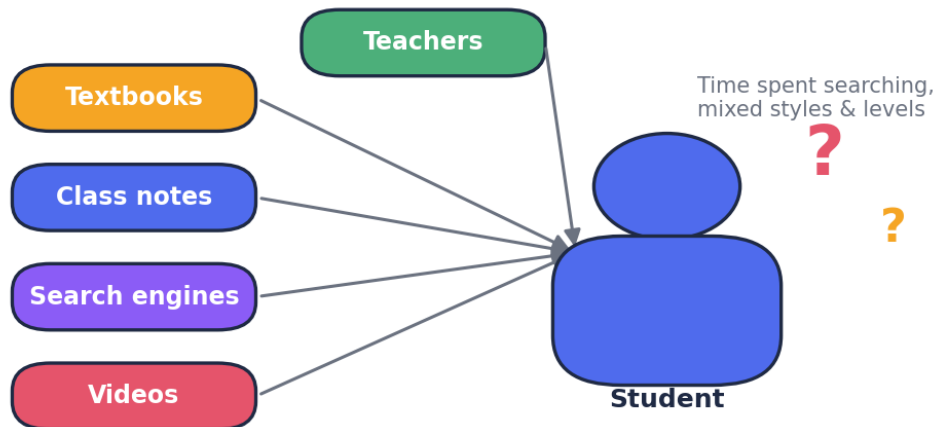


Figure 4: Existing system – information is scattered across many sources

## 5. PROPOSED SYSTEM

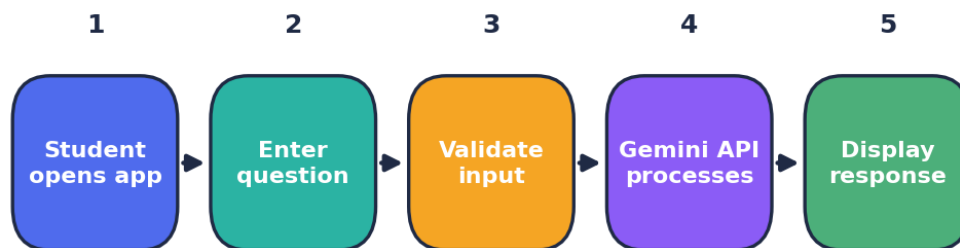
The proposed EduGenie system provides a single interface where a student can ask questions and request learning materials. The system processes the user's prompt and sends it to a Gemini model through the Gemini API.

Basic working flow:

1. Student opens EduGenie.
2. Student enters a question or selects a learning task.
3. Application validates the input.
4. Backend sends the request to Gemini using a protected API key.
5. Gemini generates a response.
6. Application receives and formats the response.
7. Student reads the explanation and continues the conversation.

Possible features include Ask a Question, Explain a Topic, Summarize, Generate Quiz, Generate Examples, and Coding Help. The exact features can be adjusted according to project requirements.

The system should clearly inform users that AI-generated answers can contain mistakes and should be verified for important academic work.



*Figure 5: EduGenie proposed workflow*

## 6. SYSTEM REQUIREMENTS

### Hardware Requirements

- Computer or laptop with at least 4 GB RAM.
- Keyboard and mouse/touchpad.
- Stable internet connection for API requests.
- Modern display and web browser.

### Software Requirements

- Operating System: Windows, Linux, or macOS.
- Frontend: HTML, CSS, JavaScript or a suitable web framework.
- Backend: Python or JavaScript/Node.js.
- Gemini SDK/API for communication with Gemini.
- Code editor such as Visual Studio Code.
- Modern web browser such as Chrome, Edge, or Firefox.

Google's current Gemini API getting-started documentation provides SDK examples for Python and JavaScript and explains API-key setup.

**Security Requirement:** The Gemini API key should not be exposed in client-side code. A backend environment variable or secure server-side configuration should be used.

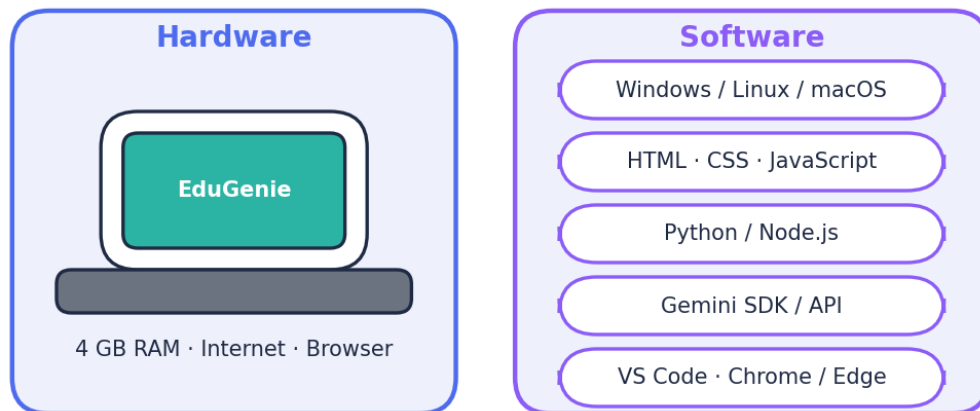


Figure 6: Hardware and software requirements

## 7. SYSTEM DESIGN AND ARCHITECTURE

EduGenie can follow a simple three-layer architecture.

**1. Presentation Layer:** This is the user interface. It contains the chat box, buttons, response area, loading indicator, and optional quiz or summary controls.

**2. Application Layer:** The backend receives the student's request, validates it, prepares the prompt, calls the Gemini API, and returns the response.

**3. AI Service Layer:** Gemini processes the prompt and generates the requested educational content.

### Text Architecture:

Student → EduGenie Interface → Backend → Gemini API → Gemini Model → Backend → EduGenie Interface → Student

The backend should handle API errors, empty prompts, timeouts, and unexpected responses. For a real deployment, authentication, rate limits, logging, and secure secret management can also be added.

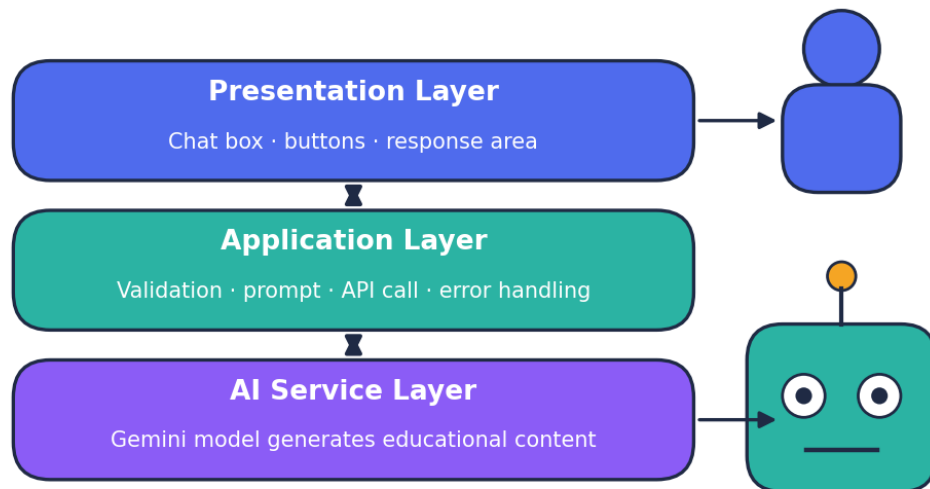


Figure 7: Three-layer architecture

## 8. MODULES OF EDUGENIE

### Module 1 - Question Answering

Students can ask general academic questions and receive explanations.

### Module 2 - Topic Explanation

The user enters a topic and selects a preferred level such as beginner, intermediate, or detailed.

### Module 3 - Summarization

The application can request a short summary of supplied text or a topic.

### Module 4 - Quiz Generator

The system can generate multiple-choice or short-answer practice questions.

### Module 5 - Example Generator

The system can provide simple examples related to a concept.

### Module 6 - Coding Assistant

Students can request beginner-friendly explanations of programming concepts and small educational examples.

### Module 7 - Conversation

Follow-up questions can be asked so the student can continue learning in the same session.

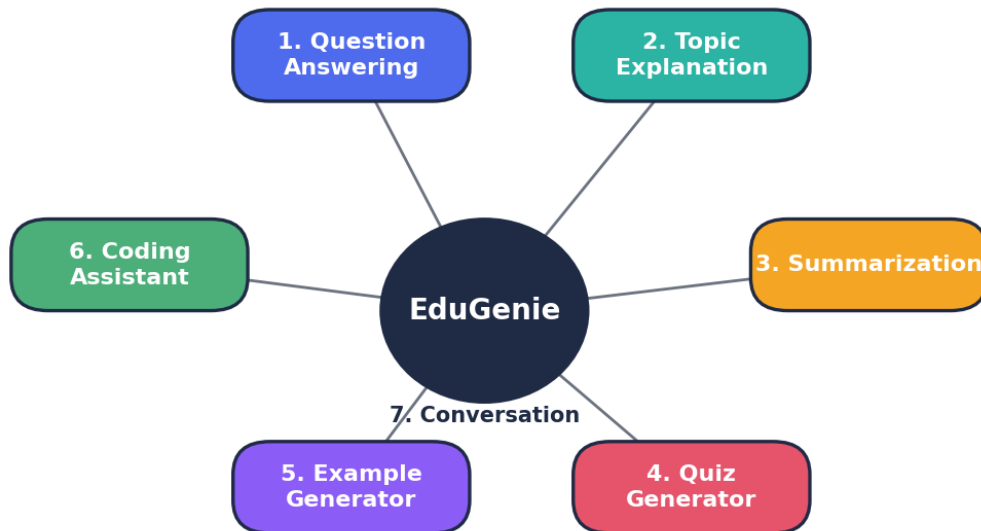


Figure 8: EduGenie modules

## 9. GEMINI API INTEGRATION

The Gemini API provides a programmatic way to integrate Gemini models into applications. Current Google documentation describes the Interactions API as the recommended standard interface for new projects, while the older generateContent interface remains supported.

A simplified Python example using the current Google GenAI SDK is shown below. The API key should be supplied through a secure environment variable.

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.8-flash",
    input="Explain binary search in simple words"
)

print(interaction.output_text)
```

The official getting-started documentation shows installation using the google-genai package and API key configuration through the GEMINI\_API\_KEY environment variable.

For a college demonstration, the project can use a simple prompt-response workflow. The exact model name should be checked against the current Gemini model documentation when the project is implemented.

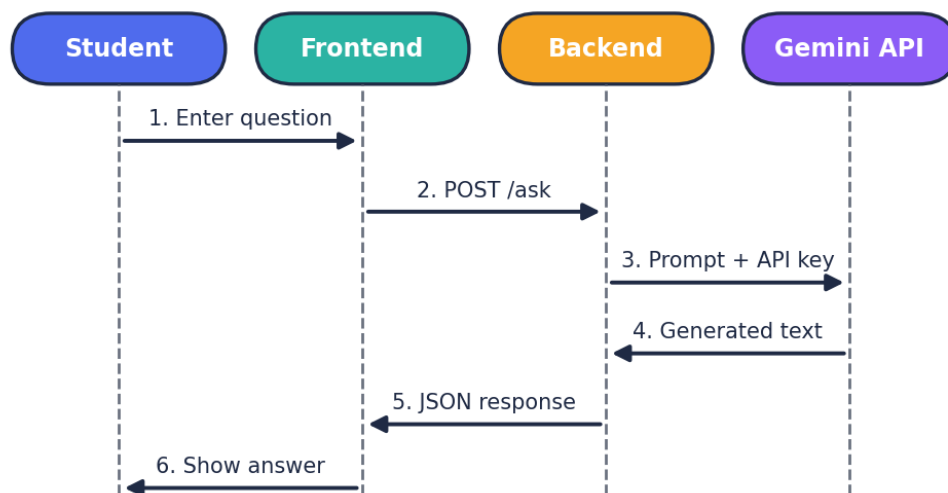


Figure 9: Request/response sequence between student, frontend, backend and Gemini API

## 10. IMPLEMENTATION

**Step 1:** Create the user interface. Add a title, question box, submit button, response area, and optional buttons for Summary and Quiz.

**Step 2:** Create the backend. The backend receives the user's prompt and performs basic validation.

**Step 3:** Configure the Gemini API. Create an API key through the appropriate Google AI developer tools and store it securely as an environment variable.

**Step 4:** Send the prompt to Gemini. The backend calls the selected Gemini model and receives the generated output.

**Step 5:** Display the answer. The frontend shows the response in a readable format.

**Step 6:** Add error handling. If the API request fails, the application should show a simple message such as “Unable to generate a response. Please try again.”

**Step 7:** Test different prompts. Test definitions, explanations, summaries, quizzes, and programming questions.

### Prompt example:

*Explain the topic “Binary Search” for a beginner. Give a short definition, simple steps, and one small example.*

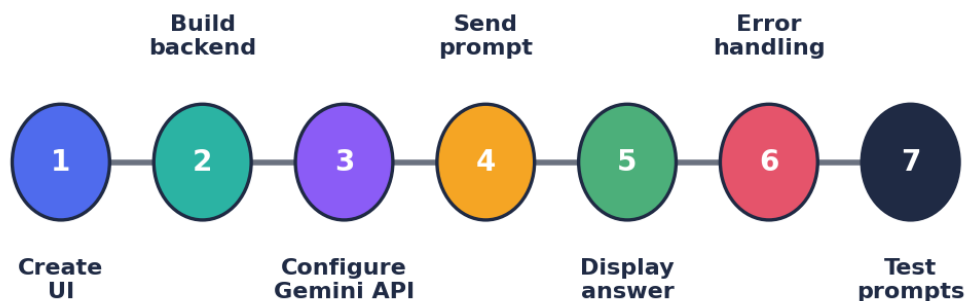


Figure 10: Implementation roadmap

# 11. USER INTERFACE AND WORKFLOW

The user interface should be simple enough for a beginner. A possible home screen contains the project name, a text input box, a Send button, and quick-action buttons.

Example workflow:

1. Open the EduGenie application.
2. Enter: "Explain stack in simple words."
3. Click Send.
4. Wait while the request is processed.
5. Read the generated explanation.
6. Ask a follow-up question if required.
7. Use the Quiz button to generate practice questions.

The interface can also provide a language option if the project team wants to support English and regional-language explanations. A clear loading message should be displayed while waiting for the API response.

The design should avoid unnecessary complexity. The main goal is to help the student ask a question and understand the response quickly.

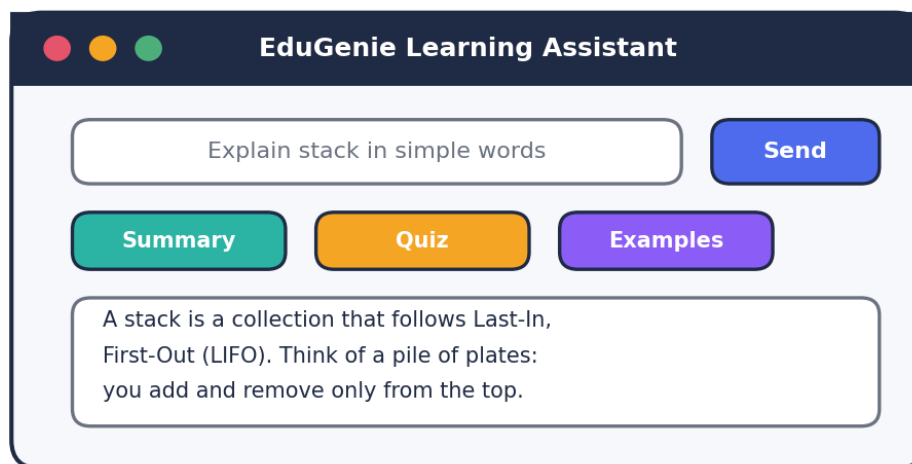


Figure 11: Sample user interface

## 12. TESTING

Testing is performed to check whether EduGenie responds correctly to different types of user inputs.

**Test Case 1:** Valid academic question – Expected result: relevant explanatory response.

**Test Case 2:** Empty input – Expected result: validation message asking the user to enter a question.

**Test Case 3:** Summary request – Expected result: shorter version of the supplied content.

**Test Case 4:** Quiz request – Expected result: practice questions in the requested format.

**Test Case 5:** Programming question – Expected result: beginner-friendly explanation or example.

**Test Case 6:** Network/API failure – Expected result: friendly error message without crashing the application.

**Test Case 7:** Very long or unusual input – Expected result: safe handling, appropriate limits, or an error message.

Testing should be repeated after changes to the frontend, backend, prompts, or API integration.

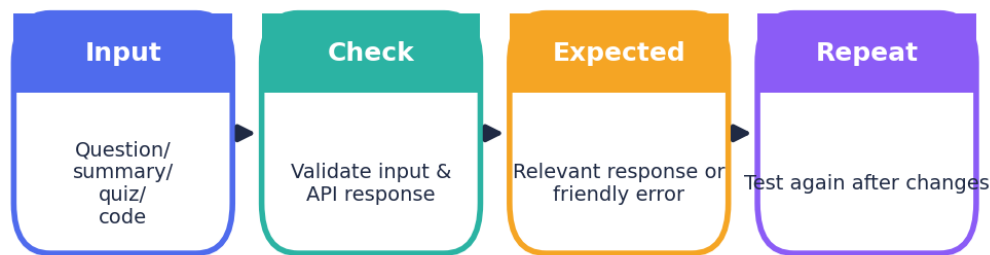


Figure 12: Testing flow

# 13. ADVANTAGES, LIMITATIONS AND FUTURE ENHANCEMENTS

## Advantages

- Easy conversational learning interface.
- Quick explanations and examples.
- Can generate revision material.
- Can support multiple academic subjects.
- Can be extended with multimodal capabilities.

## Limitations

- AI responses may contain incorrect or incomplete information.
- Internet access is normally required for API-based use.
- API usage may be subject to quotas, limits, or cost depending on the account and service.
- Response quality depends on the clarity of the prompt.
- Student learning should not depend only on AI-generated answers.

## Future Enhancements

- Student login and personalized profiles.
- Saved study history and progress tracking.
- Document or PDF-based question answering.
- Voice input and output.
- Multilingual learning support.
- Teacher dashboard and assignment support.
- Topic-wise progress and quiz scores.
- More structured study plans.

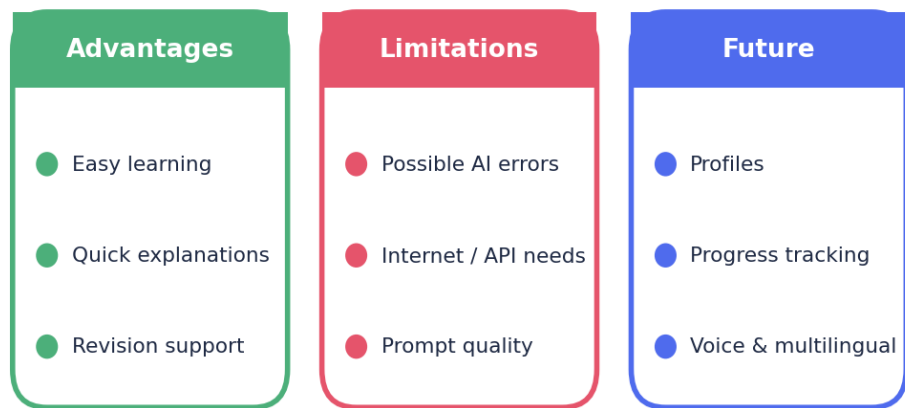


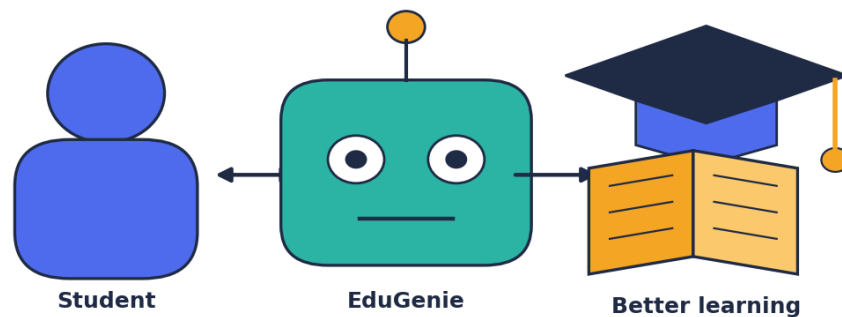
Figure 13: Project overview

# 14. CONCLUSION AND REFERENCES

## Conclusion

EduGenie demonstrates how generative AI can be integrated into an educational application to provide interactive learning assistance. The system allows students to ask questions, receive simple explanations, generate summaries and quizzes, and continue a learning conversation.

The project provides practical knowledge of application design, API integration, prompt engineering, error handling, and software testing. With appropriate verification and responsible use, an AI assistant can serve as a useful additional learning tool.



*Figure 14: Student and EduGenie working together*

## References

1. Google AI for Developers – Gemini API documentation. Official documentation covering Gemini API capabilities and integration.
2. Google AI for Developers – Getting Started with Gemini API. API-key setup and SDK examples.
3. Google AI for Developers – Gemini API Reference. API endpoints and request/response information.
4. Google AI for Developers – Interactions API Overview. Current recommended interface for new Gemini projects.